Lab 4:
Enhancing the CRUD Application with Form Validation and Error Handling

To prevent users from submitting incomplete or incorrect data (like a book with a blank title or a ridiculously long author name), we need to implement validation. In ASP.NET Core MVC, this is achieved using **Data Annotations** on the model, checking the **ModelState** in the controller, and using **Validation Tag Helpers** in the views.

Here is how to wire up robust server-side and client-side validation for your Book CRUD application.

Update the Model

Defining the rules with Data Annotations

In VS Code, open /Models/Book.cs. We will add rules directly to the model properties. ASP.NET Core uses these attributes to automatically validate incoming data and generate the appropriate error messages.

Replace the code in **Book.cs** with this updated version:

```
C#

using System.ComponentModel.DataAnnotations;

namespace MyMvcApp.Models
{
    public class Book
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "The Title field is
required.")]
        [StringLength(100, MinimumLength = 2, ErrorMessage =
"Title must be between 2 and 100 characters.")]
        public string Title { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please provide the author's
name.")]
```

```
        [StringLength(50, ErrorMessage = "Author name cannot
exceed 50 characters.")]
        public string Author { get; set; } = string.Empty;
    }
}
```

Update the Controller

Checking the ModelState before saving data

When a form is submitted, ASP.NET Core automatically checks the data against your Model's Data Annotations. The result of this check is stored in a property called ModelState.IsValid. If the data fails validation, we must reject the save operation and send the user back to the form with their original data so they can fix the errors.

Open /Controllers/BooksController.cs and update the [HttpPost] methods for **Create** and **Edit**:

C#

```
    // UPDATE ONLY the POST Create method
        [HttpPost("New")]
        public IActionResult Create(Book book)
        {
            // Error Handling: Check if the submitted data
passes our Data Annotations
            if (!ModelState.IsValid)
            {
                // Return the view immediately, passing back the
invalid book object
                // so the user doesn't lose what they already
typed.
                return View(book);
            }

            book.Id = _nextId++;
            _books.Add(book);
            return RedirectToAction("Index");
        }

        // UPDATE ONLY the POST Edit method
        [HttpPost("{id:int}/Edit")]
```

```
        public IActionResult Edit(int id, Book updatedBook)
        {
            if (id != updatedBook.Id) return BadRequest();

            // Error Handling: Validate before updating
            if (!ModelState.IsValid)
            {
                return View(updatedBook);
            }

            var book = _books.FirstOrDefault(b => b.Id ==
updatedBook.Id);
            if (book != null)
            {
                book.Title = updatedBook.Title;
                book.Author = updatedBook.Author;
            }
            return RedirectToAction("Index");
        }
```

Update the Views

Displaying validation errors to the user

Now we need to show the user exactly what went wrong. We will add a global validation summary at the top of the form, and specific error messages next to each input field using Tag Helpers.

Open /Views/Books/Create.cshtml and replace the content with this code:

```
HTML

@model MyMvcApp.Models.Book

<h2>Add a Book</h2>

<form asp-action="Create" method="post">
    <!-- Global summary for form-level errors -->
    <div asp-validation-summary="ModelOnly" class="text-danger
mb-3"></div>

    <div class="mb-3">
```

```
        <label asp-for="Title"></label>
        <input asp-for="Title" class="form-control" />
        <!-- Displays the specific error message for Title -->
        <span asp-validation-for="Title"
class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Author"></label>
        <input asp-for="Author" class="form-control" />
        <!-- Displays the specific error message for Author -->
        <span asp-validation-for="Author"
class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-success">Save</button>
</form>

<!-- This script section enables fast client-side (JavaScript)
validation -->
@section Scripts {
    @{await
Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

Open /Views/Books/Edit.cshtml and apply the same validation tags:

HTML

```
@model MyMvcApp.Models.Book

<h2>Edit Book</h2>

<form asp-action="Edit" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger
mb-3"></div>
    <input type="hidden" asp-for="Id" />

    <div class="mb-3">
        <label asp-for="Title"></label>
        <input asp-for="Title" class="form-control" />
```

```
        <span asp-validation-for="Title"
class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Author"></label>
        <input asp-for="Author" class="form-control" />
        <span asp-validation-for="Author"
class="text-danger"></span>
    </div>

    <button type="submit" class="btn
btn-primary">Update</button>
</form>

@section Scripts {
    @{await
Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

Test the Application

Here is a structured testing script you can include in your lab manual to help students verify their work. It guides them through testing the constraints and proves the difference between client-side and server-side validation.

Test 1: Testing the Validation Rules

Now that the validation is wired up, let's break the application on purpose. Follow these steps to verify that your rules are protecting the data.

Test 1: The Empty Submission (Testing [Required])

1. Run your app and navigate to **Add New Book**.
2. Do not type anything into the fields.
3. Click the **Save** button.

- **Expected Result:** The form should not submit. You should instantly see "The Title field is required." and "Please provide the author's name." in red.
- *Notice how fast this happens without the page reloading. This is your JavaScript Client-Side validation at work.*

Test 2: Minimum Length Constraints

1. In the **Title** field, type a single letter (e.g., "A").
2. Click **Save** or just click outside the text box.

- **Expected Result:** The error message under Title should change to: "Title must be between 2 and 100 characters."

Test 3: The "Hacker" Test (Testing Server-Side Validation)

Client-side validation is great for user experience, but it can be bypassed. Let's prove your C# Controller is actually protecting the server.

1. While on the **Add New Book** page, press F12 to open your browser's Developer Tools.
2. Press F1 (or go to Settings) and check the box to **Disable JavaScript**.
3. Refresh the page. (The client-side validation script is now broken).
4. Leave the form completely empty and click **Save**.

- **Expected Result:** The page *will* reload this time (because the browser sent the HTTP request), but the server will still reject it. Your C# ModelState.IsValid check caught the empty data, and the server returned the form with the red error messages.
- *Note: Don't forget to re-enable JavaScript when you are done!*

Test 4: Valid Data (The Happy Path)

1. Fill out the form correctly: Title = "Dune", Author = "Frank Herbert".
2. Click **Save**.

- **Expected Result:** The data passes validation, saves to the list, and redirects you back to the Index page where you can see your new book.

Test 5: Edit View Validation

1. On the Index page, click **Edit** next to an existing book.
2. Delete the Author name so the field is completely empty.

3. Click **Update**.

- **Expected Result:** Validation errors should appear here just as they did on the Create page, proving your Edit.cshtml view is also properly protected.

Theoretical Background

Form Validation and Error Handling Theory

Robust applications must never trust user input. Validation ensures that data entering your system conforms to expected formats, preventing database errors, security vulnerabilities, and logic bugs. ASP.NET Core utilizes a unified validation model that handles both the server and the browser automatically.

Server-Side vs. Client-Side Validation

Validation Layer	Execution Location	Purpose
Client-Side	Browser (via JavaScript)	Provides instant feedback to the user without a page reload, improving UX. It prevents unnecessary network traffic but **can be easily bypassed** by malicious users disabling JavaScript or using API tools like Postman.
Server-Side	Server (via C# Controller)	The ultimate source of truth. It checks the data after it arrives over the network. This layer is mandatory for security and data integrity, as it cannot be bypassed by the client.

Data Annotations and ModelState

ASP.NET Core links these two layers using **Data Annotations**. By placing attributes like [Required], [EmailAddress], or [Range] on your C# Model properties, the framework automatically generates the required HTML5 and JavaScript attributes for the View, while simultaneously enforcing the rules in the Controller.

ModelState is a dictionary object within the Controller that represents the result of the data binding and validation process.

- When a POST request arrives, the framework maps the incoming form data to the Model.

- It evaluates all Data Annotations.

- If any rule fails, it flips ModelState.IsValid to false and populates the dictionary with the specific property names and their corresponding error messages.

- The developer checks if (!ModelState.IsValid) and returns the View. The Tag Helpers (asp-validation-for) then extract those exact messages from the ModelState dictionary and inject them directly into the HTML to show the user what needs fixing.